

Практическая работа по Информатике

Файлы

Для работы с файлом из программы требуется его открыть. При этом программа обращается к операционной системе с запросом, который содержит имя файла, а также режим работы с файлом. Для хранения этих параметров нужны специальные файловые переменные. Также необходимо определиться с типом файла. В первом примере у нас тип файла text, имя файловой переменной f с помощью команды assign связывается с именем файла на диске. А с помощью процедуры rewrite выбирается режим работы «чтение-запись» с этим файлом.

Первый вариант программы находится в файле /home/method/hellofile.pas, скопируйте его к себе в домашний каталог, скомпилируйте и проверьте в работе. Недостаток программы hellofile состоит в отсутствии обработки ошибок при обращении к файловой системе.

В программе /home/method/hellofiler.pas добавлена обработка ошибки сначала при открытии файла, а затем – при записи в файл:

```
program HelloFileR;
const
  message = 'Hello, world!';
  filename = 'hello.txt';
var
  f: text;
begin
  {$I-}
  assign(f, filename);
  rewrite(f);
  if IOResult <> 0 then
  begin
    writeln('Couldn't open file ', filename);
    halt(1)
  end;
  writeln(f, message);
  if IOResult <> 0 then
  begin
    writeln('Couldn't write file');
    halt(1)
  end;
  close(f)
end.
```

Задание 1.

Исключите из текста программы hellofiler.pas операторы досрочного завершения программы halt, чтобы текст программы соответствовал принципам структурного программирования.

Следующий пример программы также работает с текстовым файлом, но теперь мы записываем в текстовый файл не строку, а целые числа типа Longint. При этом компилятор автоматически преобразует числовые данные в символы и строки. Запустите программу и посмотрите содержимое файла numbers.txt. Исходный текст программы также можно скопировать из /home/method/gennumtx.pas.

```
program GenerateNumTxt;
```

```

const
    name = 'numbers.txt';
    start = 1000;
    step = 1001;
    count = 100;
var
    f: text;
    i: integer;
    n: longint;
begin
    assign(f, name);
    rewrite(f);
    n := start;
    for i := 1 to count do
        begin
            writeln(f, n);
            n := n + step
        end;
    close(f)
end.

```

Задание 2.

Поменяйте тип файла с текстового на типизированный файл целых чисел типа Longint в программе GenerateNumTxt.pas, измените имя программы на GenerateNumBin.pas, а имя создаваемого файла – на numbers.bin. Добавьте обработку ошибок сначала при открытии файла, а затем – при записи в файл. Проверить результат можно с помощью утилиты hexdump с ключом -d: hexdump -d numbers.bin

Третий пример познакомит нас с блочным вводом-выводом на уровне файла. Также этот метод называют работой с нетипизированными файлами. Его применяют, когда требуется работать с большими порциями данных. Тип файловой переменной для блочного ввода-вывода называется file. Для связывания файловой переменной и имени файла на диске здесь также используется процедура assign. Открываются файлы также с помощью reset и rewrite, но у этих процедур при работе с нетипизированными файлами добавляется второй параметр – целое число, означающее размер блока.

Для чтения из нетипизированных файлов и записи в них используются процедуры BlockRead и BlockWrite, которые очень похожи друг на друга, обе получают по четыре параметра:

- 1) файловая переменная;
- 2) переменная произвольного типа и размера, в которую будет помещена информация, прочитанная из файла, либо из которой будет взята информация для записи в файл;
- 3) целое число, задающее количество блоков, которые следует соответственно прочитать или записать;
- 4) переменная типа longint, int64, word или integer, в которую процедуры записывают реальное количество прочитанных или записанных блоков.

Для примера рассмотрим программу BlockFileCopy, которая копирует один файл в другой, получив их имена из командной строки. Как для исходного, так и для целевого файлов (англ. source и destination) мы будем использовать нетипизированные файловые переменные; первый файл мы откроем в режиме «только чтение», второй – в режиме «только запись». Чтение мы будем производить порциями по 4096 байт (4KB), причём этот размер вынесем в константу; в

качестве буфера, то есть переменной, в которую помещается прочитанная информация, мы воспользуемся массивом соответствующего размера из элементов типа byte.

Записывать в целевой файл мы будем на каждом шаге ровно столько информации, сколько было прочитано из исходного. При возникновении ситуации «конец файла» мы немедленно завершим цикл чтения/записи, причём сделать это нам придётся до выполнения записи, то есть из середины цикла; мы воспользуемся для этого оператором break, а сам цикл сделаем «бесконечным». После завершения цикла мы должны будем, естественно, закрыть оба файла. Поскольку нам известно о существовании переменной ErrOutput, обозначающей поток сообщений об ошибках, все такие сообщения будем выдавать, как положено, в этот поток. Завершать программу после обнаружения ошибок будем с кодом 1, чтобы показать операционной системе, что у нас что-то пошло не так.

```
program BlockFileCopy;
const
  BufSize = 4096;
var
  src, dest: file;
  buffer: array [1..BufSize] of byte;
  ReadRes, WriteRes: longint;
begin
  {$I-}
  if ParamCount < 2 then
    begin
      writeln(ErrOutput, 'Expected: source and dest. names');
      halt(1)
    end;
  assign(src, ParamStr(1));
  assign(dest, ParamStr(2));
  filemode := 0;
  reset(src, 1);           { block size of 1 byte }
  if IOResult <> 0 then
    begin
      writeln(ErrOutput, 'Couldn''t open ', ParamStr(1));
      halt(1)
    end;
  filemode := 1;
  rewrite(dest, 1);
  if IOResult <> 0 then
    begin
      writeln(ErrOutput, 'Couldn''t open ', ParamStr(2));
      halt(1)
    end;
  while true do
    begin
      BlockRead(src, buffer, BufSize, ReadRes);
      if ReadRes = 0 then    { end of file! }
        break;
      BlockWrite(dest, buffer, ReadRes, WriteRes);
      if WriteRes <> ReadRes then
        begin
          writeln(ErrOutput, 'Error writing the file');
          break
        end
      else
        break
      end
    end
  end;
```

```
        end  
    end;  
    close(src);  
    close(dest)  
end.
```

Задание 3.

Скопируйте исходный текст программы из /home/method/block_cp.pas к себе в домашний каталог. Скомпилируйте программу, затем изучите работу с блочным вводом-выводом на примере этой программы.